

Refactoring

Note: This page summarizes James Shore's article on Refactoring from *The Art of Agile Development*.

Improving Code Structure Without Changing Behavior

Entropy always wins. Eventually, even beautifully designed code becomes a mess. Refactoring is how you fight back.

Refactoring is the process of changing code's design without changing its behavior. What the code does stays the same; how it does it changes. Like factoring $x^2 - 1$ into $(x + 1)(x - 1)$, you're restructuring without altering meaning.

How to Refactor

Refactoring isn't rewriting. You don't just change the design—you make changes in a series of tiny, controlled steps. Each step should take seconds to minutes. Your tests should pass after each one.

Think of it like turns on a Rubik's cube. Each individual refactoring is small. To achieve anything significant, you string many together. But at any moment, you're in control and can stop.

The key is small steps. When something breaks, there's almost nothing to check. You remain in control of your code at all times.

Code Smells

Code smells are patterns that suggest design problems—like a funny smell in the kitchen that makes you look closer. They don't necessarily mean something's wrong, but they're worth investigating.

Common smells include:

Divergent Change — One class changes for multiple unrelated reasons. Split it so each concept has its own home.

Shotgun Surgery — One change requires modifying multiple classes. The concept is scattered; consolidate it.

Primitive Obsession — High-level concepts represented with primitive types. A `decimal` for dollars or strings for addresses should probably be their own classes.

Data Clumps — Variables that travel together. If several values always appear as a group, they probably belong in a class.

Data Class — A class with only data and getters/setters. Data and behavior belong together—move methods to where the data lives.

Time Dependencies — Methods that must be called in a specific order. This usually indicates an encapsulation problem.

Coddling Nulls — Passing nulls around and constantly checking for them. Adopt a fail-fast strategy instead; throw exceptions rather than returning null.

When to Refactor

Refactor constantly. Perform small refactorings during TDD. Do bigger refactorings regularly. Every week, your design should be better than it was the week before.

You don't need to refactor code unrelated to your current work. But when you touch code, leave it cleaner than you found it.

Prerequisites

Refactoring requires good tests. Without them, you can't tell if you've accidentally changed behavior. If you're working with untested legacy code, write a few end-to-end tests first to create a safety net.

Refactoring also requires collective code ownership and continuous integration. Design changes touch many parts of the codebase. You need permission to change anything and the ability to integrate frequently.

There Is No Alternative

No matter how carefully you design, all code accumulates technical debt. Without refactoring, that debt eventually overwhelms you. Your choices become rewriting (expensive and risky) or abandoning the software entirely.

Refactoring is how you pay down debt continuously, keeping entropy at bay.

References

- James Shore - Refactoring — The primary source for this section, from *The Art of Agile Development*.
- Martin Fowler, *Refactoring: Improving the Design of Existing Code* (2018, 2nd ed.) — The definitive reference. Essential reading.
- Joshua Kerievsky, *Refactoring to Patterns* (2004) — How individual refactorings combine to achieve significant design changes.